The API you're creating for Agtron's shop is a thin wrapper for accessing some information in key-value store databases. The databases have two operations, `set` and `get`. The `set` operation takes a key, a value, and a callback function. The `get` operation takes a key and a callback. For example, the user data is loaded from the database in the same way that product data is loaded from the database:

```
users.get parts[2], (error, user) ->
  res.end JSON.stringify user, 'utf8'

products.get parts[2], (product) ->
  res.end JSON.stringify product, 'utf8'
```

There's no meaningful name describing what these sections of the program do, so you start by naming them:

```
loadUserData = (user, callback) ->
  users.get user, (data) ->
    callback data

loadProductData = (product, callback) ->
  products.get product, (data) ->
    callback data
```

If you didn't notice before that the code was repetitive, you definitely notice now. The `saveUserData` and `saveProductData` functions are almost exactly the same.

Instead of having variations of the same basic function appear repeatedly in your program, create an abstraction and eliminate the duplication:

```
makeLoadData = (db) ->
  (entry, callback) ->
    db.get entry, (data) ->
      callback data

makeSaveData = (type) ->
  (entry, value, callback) ->
    db.set entry, value, callback?()
```

Putting the existential operator in front of a callback prevents an error from occurring if the callback argument is not passed in when the function is invoked. You'll see more about advanced use of the existential operator in the next chapter.

Both of these functions return functions. What are they used for? With these two functions you can create a `loadUserData` function and a `saveUserData` function: